CODD'S RULE

Rule 1: Information Rule - All data must be represented as values in tables (relations).

A DBMS should store all information only in the form of tables, not in pointers, files, or structures.

Example:
Instead of storing student data in files or objects:

STUDENT(StudentID, Name, Dept, CGPA)

All information (Name, Dept, CGPA) appears as **table values**, not embedded in program code.

Rule 2: Guaranteed Access Rule - Every data value must be accessible using table name, primary key, and column name.

There should be no ambiguity in accessing data.

Example:
To access CGPA of student with ID = 101:

SELECT CGPA FROM STUDENT WHERE StudentID = 101;

✔ Table name: STUDENT
✔ Key: StudentID
✔ Column: CGPA

Rule 3: Systematic Treatment of NULL Values - NULLs must be supported and handled uniformly.

NULL represents missing, unknown, or inapplicable data—not zero or empty string.

Example:
If a student has not declared a minor subject:

STUDENT(StudentID, Name, Minor)

101, "Anu", NULL

DBMS distinguishes between:

- NULL (unknown)
- 0 (numeric value)
- "" (empty string)

Rule 4: Dynamic Online Catalog (Data Dictionary) - Metadata must be stored as tables and be queryable using SQL.

Users should be able to query schema information like normal data.

Example:
To view table details:

SELECT * FROM INFORMATION_SCHEMA.TABLES;

Here, table definitions are stored as relations, not hidden files.

Rule 5: Comprehensive Data Sublanguage Rule - A single language must support data definition, manipulation, integrity, and authorization.

SQL should support:

- DDL (CREATE, ALTER)
- DML (SELECT, INSERT)
- Integrity constraints
- Authorization

Example:

CREATE TABLE STUDENT (...);

INSERT INTO STUDENT VALUES (...);

GRANT SELECT ON STUDENT TO user1;

Rule 6: View Updating Rule - All theoretically updatable views must be updatable.

Users should be able to insert/update data through views when logically possible.

Example:

CREATE VIEW CS_STUDENTS AS

SELECT * FROM STUDENT WHERE Dept = 'CSE';

INSERT INTO CS_STUDENTS VALUES (105, 'Ravi', 'CSE', 8.2);

✔ DBMS should update the base table automatically.

Rule 7: High-Level Insert, Update, Delete - Operations must work on sets of rows, not just single rows.

SQL supports set-oriented operations.

Example:

UPDATE STUDENT

SET CGPA = CGPA + 0.2

WHERE Dept = 'AI&DS';

✔ Updates **multiple records at once**

Rule 8: Physical Data Independence - Changes in physical storage should not affect applications.

Changing storage (indexes, file organization) should not break queries.

Example:
Adding an index:

CREATE INDEX idx_cgpa ON STUDENT(CGPA);

Rule 9: Logical Data Independence - Changes in logical structure should not affect user views.

Explanation:
Users should not be affected by table restructuring.

Example:
Splitting table:

STUDENT → STUDENT_BASIC + STUDENT_ACADEMIC

Views can hide the change:

CREATE VIEW STUDENT_VIEW AS

SELECT ... FROM STUDENT_BASIC JOIN STUDENT_ACADEMIC;

✔ Old queries still function.

Rule 10: Integrity Independence - Integrity constraints must be stored in the catalog, not application code.

Constraints like primary key, foreign key, and checks should be managed by DBMS.

Example:

CREATE TABLE STUDENT (

StudentID INT PRIMARY KEY,

CGPA FLOAT CHECK (CGPA <= 10)

);

✔ Rules enforced centrally by DBMS.

Rule 11: Distribution Independence - Users should not know whether data is distributed or centralized.

Data may be stored across locations, but access remains same.

Example:
Query:

SELECT * FROM STUDENT;

✔ Works whether STUDENT table is:

- On one server
- Split across multiple servers

Rule 12: Non-Subversion Rule - Low-level operations must not bypass integrity rules.

No way should exist to violate constraints using internal access.

Example:
Even if accessing data through internal APIs or bulk loaders, DBMS should enforce:

- Primary key
- Foreign key
- Check constraints

✔ Integrity rules **cannot be bypassed**

Rule No	**Rule Name**
1	Information Rule
2	Guaranteed Access
3	Systematic NULL Handling
4	Dynamic Online Catalog
5	Comprehensive Data Language
6	View Updating
7	High-Level Operations
8	Physical Data Independence
9	Logical Data Independence
10	Integrity Independence
11	Distribution Independence
12	Non-Subversion